

CS4.406 Assignment 2: Design Note

Learning from Click-Logs on EB-NeRD and MIND

What Was Built

A two-stage retrieve-then-rank news recommender, evaluated on EB-NeRD and MIND, plus a reproduction of the NRMS baseline and one improvement to it: a freshness term that gives the model a channel for how old an article is when shown.

1 Data Pipeline

Both datasets are parsed into one schema: an articles table (title, abstract, body, category, entities, published_time) and a behaviors table (one row per impression: history, candidates, labels, timestamp). EB-NeRD additionally carries session_id and per-history-click dwell times; MIND has neither, and that asymmetry recurs throughout the results.

Splits are temporal rather than random, to prevent future leakage during training. Article features such as freshness, popularity and CTR are computed strictly from data before the impression time t , and within-session features count only impressions that occurred earlier in the same session.

2 Stage 1: Candidate generation

2.1 The Problem

Assignment 1 retrieved the top-k similarity between the user's click history and the whole catalogue. We used the same pipeline to generate the top-200 candidates but the recall was very low, 2.55% (EB-NeRD) and 2.90% (MIND). The best A1 configuration reached 4.39% (EB-NeRD, BM25), and that too on the larger `ebnerd_small` bundle; on the demo bundle we run here the same pipeline gives 2.55%.

2.2 The fix

The ceiling was not in the data. It came from where stage 1 searched and what it ranked by. Clicks go to whatever is in circulation at that moment. EB-NeRD's median clicked article is 3.1 hours old when it is clicked, and 92% are clicked within a day of publication. A1 searched the full catalogue of 11,777 articles, spanning publication dates from 2000 to 2023, where 17.2% were published over a year before the test period. Ranking a week's worth of clicks against two decades of articles is the wrong search space. Two mistakes compounded.

1. The search space was wrong. We restricted A1's BM25, unchanged, to the articles the platform displayed in the previous hour. EB-NeRD recall@200 goes from 2.55% to 84.2% on the scoring alone but for MIND the recall was 21.0% so the ranking signal was not accurate.

2. The ranking signal was wrong. Ranking that same pool by recent popularity instead of history similarity reaches 97.0% and 93.9% (EB-NeRD and MIND). History similarity adds nothing on top of it.

The pool uses only events strictly before the impression time t : displays in $[t - 1h, t)$, and clicks from impressions in $[t - 1h, t - 10min)$, because a click lands after the impression that produced it. If the one-hour pool holds fewer than K articles we backfill from the previous 24 hours. We picked the window by measuring it: on 2,000-impression samples, 1 hour beat both 6 and 24 hours (EB-NeRD 97.4% against 88.9% @200), because longer windows let stale but popular articles crowd out fresh ones.

Arm	EB-NeRD @50/@100/@200	MIND @50/@100/@200
bm25, A1 over the whole catalogue	0.82 / 1.41 / 2.55%	1.27 / 1.96 / 2.90%
bm25_fresh, A1 scoring on a 1 h pool	21.9 / 42.9 / 84.2%	10.0 / 14.2 / 21.0%
popular, 1 h pool with lagged clicks	83.4 / 93.8 / 97.0%	81.6 / 88.6 / 93.9%

Table 1: Test recall@K over full populations. The bm25 row reproduces our own rerun of the A1 pipeline on the A2 splits, so the three arms are like-for-like.

3 Stage 2: The Re-ranker

3.1 The Objective

LambdaMART: Uses the Gradient Boosting Trees (MART) and the LambdaRank optimization which effectively learns to rank items which will improve the metrics like nDCG. It is a *pairwise gradient which weighted by listwise metric*.

We set *lambdarank_truncation_level* to 200, the full group size, so that the full-group setting makes the $|\Delta \text{nDCG}|$ weights reflect the whole 200-item list the metrics are computed over.

3.2 The features

Used **24** features for training this model.

Click history: click count, recency-weighted engagement, has_history_timestamps, avg history dwell time + availability flag

Session: hour, day-of-week, impression size, session position, impressions/clicks earlier in session

Article: click count, display count, log click count, smoothed CTR, log freshness, has_known_publish_time.

history $\times$ candidate: category binary match, category affinity, history_title, history_embedding

From Stage 1: retrieval rank, position bias, retrieval score

3.3 Results

	EB-NeRD		MIND	
	stage 1 $\rightarrow$ re-ranked	Δ (95% CI)	stage 1 $\rightarrow$ re-ranked	Δ (95% CI)
AUC	0.8815 $\rightarrow$ 0.9219	+0.0404 [+0.0391, +0.0416]	0.8655 $\rightarrow$ 0.8891	+0.0235 [+0.0227, +0.0244]
MRR	0.1639 $\rightarrow$ 0.2115	+0.0476 [+0.0442, +0.0510]	0.2237 $\rightarrow$ 0.2403	+0.0167 [+0.0149, +0.0183]
nDCG@5	0.1471 $\rightarrow$ 0.2026	+0.0555 [+0.0515, +0.0595]	0.2451 $\rightarrow$ 0.2618	+0.0166 [+0.0146, +0.0186]
nDCG@10	0.1994 $\rightarrow$ 0.2705	+0.0711 [+0.0674, +0.0747]	0.2932 $\rightarrow$ 0.3150	+0.0217 [+0.0201, +0.0233]

Table 2: Test split, popular candidates, $K = 200$. Paired bootstrap over impressions (2,000 resamples, seed 0), both models held fixed. Every interval excludes zero.

4 Baseline reproduced, then improved (Q3)

4.1 Baseline

We reproduced the NRMS setup according to their paper (Wu et al., 2019).

4.2 The Improvement: freshness weighting

NRMS reads only title tokens, so it has no channel at all for how old an article is when shown. Based on the weightage given to the freshness feature in our re-ranker, we hypothesized that freshness will play a key role here as well. We added an age term to the NRMS score:

$$\text{score}(u, c) = \langle \mathbf{u}, \mathbf{c} \rangle + g(\log(1 + \text{age_hours}), \mathbb{I}_{\text{known}}) \quad (1)$$

where g is a $2 \rightarrow 16 \rightarrow 1$ MLP and $\mathbb{I}_{\text{known}}$ marks whether the article’s age is available. EB-NeRD ages come from `published_time`. MIND has no publish dates, so we use the first time the article appears in the display log. The head adds 65 parameters to models. The term is added to the logit rather than concatenated into the news vector, which leaves the NRMS dot-product scorer unchanged.

4.3 Results

	EB-NeRD base $\rightarrow$ +fresh	Δ (95% CI)	MIND base $\rightarrow$ +fresh	Δ (95% CI)
AUC	0.5425 $\rightarrow$ 0.6112	+0.0686 [+0.0662, +0.0710]	0.6040 $\rightarrow$ 0.6042	+0.0002 [−0.0006, +0.0010]
MRR	0.3394 $\rightarrow$ 0.3962	+0.0567 [+0.0538, +0.0595]	0.2750 $\rightarrow$ 0.2743	−0.0007 [−0.0014, +0.0001]
nDCG@5	0.3770 $\rightarrow$ 0.4427	+0.0657 [+0.0628, +0.0685]	0.2961 $\rightarrow$ 0.2949	−0.0012 [−0.0021, −0.0003]
nDCG@10	0.4570 $\rightarrow$ 0.5095	+0.0525 [+0.0501, +0.0548]	0.3617 $\rightarrow$ 0.3610	−0.0007 [−0.0015, +0.0001]

Why MIND shows nothing: The age term is additive on the logit, so it can only reorder articles *within* an impression, and MIND’s impressions are near-uniform in age (median within-impression spread **27.5 h**, against **1,057 h** on EB-NeRD), so a term that is nearly constant across a row cannot reorder it. It is not a data gap (99.9% of MIND candidates and clicks carry an age) and not the first-seen proxy either: EB-NeRD has both age definitions, and swapping real publish times for the proxy costs only **0.013** AUC (0.6765 $\rightarrow$ 0.6631). The defensible claim is thus “freshness-as-first-seen does not help on MIND”, not “freshness does not help”.

5 Serving and scale

Measured locally: components built in isolated processes, so each figure is the exact array bytes where walkable and resident-set growth otherwise. Latency is single-threaded on an AMD Ryzen 9 8940HX (16 cores, 14 GiB RAM), 1,000 timed requests after 50 warm-up, $K = 200$. Stage-1 log replay (0.02–0.03 ms/impression) is excluded: a live system maintains those counters as events arrive.

5.1 Index memory

Component	EB-NeRD	MIND
<i>ANN index and feature store</i>		
Embedding matrix, ANN copy (normalised float32)	34.5	95.6
Embedding matrix, lookup copy (user pooling)	34.5	95.6
Article category lookup	35.8	61.8
<i>Other resident state</i>		
BM25 inverted index (history-title feature)	4.1	36.0
As-of article statistics	4.8	68.1
Fresh-pool event log	7.5	26.5
LightGBM booster (174 / 168 trees)	4.0	3.9
Total (MiB)	125.2	387.4

Table 3: MiB resident. The fresh-pool log is an upper bound: it spans every split so the benchmark can replay time, whereas a live system retains only the one-hour window.

Our ANN index is a brute-force exact cosine scan, so it adds no structure of its own beyond the matrix it scans: both catalogues are under 100k articles, where a single matrix multiply is exact and faster than an approximate index. The matrix is however resident *twice*: **BruteForceANN** stores its own normalised copy while the article $\rightarrow$ vector lookup that pools the user representation keeps

the original. Normalising in place would halve it, since the vectors are already L2-normalised when the feature store is built. The feature store dominates, at 84% of EB-NeRD’s footprint and 65% of MIND’s, and the two structures serving the history-similarity features cost 227.1 MiB on MIND, which buys a significant ranking gain there but none on EB-NeRD.

5.2 Latency

	Stage 1	Features	Scoring	Total p50	Total p99
EB-NeRD	0.12	3.42	2.58	5.97	10.50
MIND	0.56	2.72	0.14	3.47	8.00

Table 4: Milliseconds for a single user request, single core, $K = 200$. Stage 1 is candidate generation, features and scoring are re-ranking.

Feature building is 57–78% of the median request and stage 1 is 2–16%. Neither retrieval nor the GBDT is the bottleneck, so latency work belongs in feature assembly.

5.3 Cost per 1000 queries at the SLA

p99 is roughly $10\times$ inside the 100 ms SLA on both datasets. At 170 (EB-NeRD) and 253 (MIND) single-core QPS and \$0.17 per vCPU-hour, that is \$0.000278 and \$0.000186 per thousand queries, under a dollar of compute for a million requests a day.

6 Where the pipeline breaks at $10\times$

Latency is linear in K with a fixed per-impression cost of about 1.2 ms (EB-NeRD 2.40 / 3.50 / 5.88 ms mean at $K = 50/100/200$; roughly $1.2 + 0.023K$). K is therefore the throughput knob, and on EB-NeRD halving it costs little recall (93.8% at $K = 100$ against 97.0% at $K = 200$; MIND 88.6% against 93.9%).

Ranked by what fails first:

1. **As-of article statistics: the binding constraint.** It must be resident and grows with total events: MIND’s 8.93M display and click events occupy 68.1 MiB at 8 bytes each, so about 680 MiB at $10\times$ traffic and about 4.4 GB at MINDlarge scale ($\sim 65\times$). This is the first thing to shard or move to a windowed counter store.
2. **CPU for feature assembly.** 170–253 QPS per core means about ten cores at $10\times$. Parallel across requests, so a cost line rather than a wall.
3. **The ANN index and feature store, on a different axis.** Both grow with *catalogue size*, not traffic, so $10\times$ traffic does not touch them, but a $10\times$ catalogue does, and it is the point where the brute-force scan stops paying and the embedding matrix needs a real approximate index instead of a resident array.

7 Beyond accuracy (Q5)

Coverage, diversity and novelty on the top-10 lists, over a 5,000-impression sample with 1,000 bootstrap resamples, plus two slices.

Raw diversity is not comparable across datasets. It is dominated by how anisotropic the embedding space is: a random EB-NeRD pair already scores 0.048 under the provided multilingual BERT vectors (cosine 0.95, a narrow cone), against 0.940 for MiniLM on MIND. Read the ratio to that baseline, where 1.0 is “as diverse as random”. EB-NeRD sits at **0.765** and MIND at **0.958**, so EB-NeRD’s recommendations concentrate more, the same direction the raw values suggest, but a $1.25\times$ difference rather than the $24\times$ they imply. Normalising changes the severity, not the ordering: 0.037 reads as near-total homogeneity, while 0.765 of chance is a mild concentration.

Metric (top-10)	EB-NeRD	MIND
Catalogue coverage	0.0696 [0.0690, 0.0701]	0.0065 [0.0063, 0.0067]
catalogue size	11,777	65,238
Diversity (raw)	0.0370 [0.0363, 0.0378]	0.9008 [0.8997, 0.9019]
random baseline	0.0484	0.9402
ratio to random	0.765	0.958
Novelty ($-\log_2 p$)	14.456	17.339

Coverage is low on both, and that is the price of Stage 1. Recommending from a one-hour display window can only surface what was in circulation, so MIND reaches 0.65% of its catalogue. The fresh pool that fixed recall also guarantees a small, recent, popular recommendation set.

	Slice	n	MRR (95% CI)
EB-NeRD	cold start (≤ 5 clicks)	67	0.2678 [0.2005, 0.3447]
	warm	25,289	0.2114 [0.2079, 0.2146]
	head article clicked	197	0.0072 [0.0062, 0.0082]
	tail article clicked	25,159	0.2131 [0.2099, 0.2165]
MIND	cold start (≤ 5 clicks)	12,982	0.2426 [0.2375, 0.2476]
	warm	60,170	0.2399 [0.2375, 0.2421]
	head article clicked	9,150	0.1613 [0.1568, 0.1661]
	tail article clicked	64,002	0.2516 [0.2494, 0.2541]

Table 5: Head is the top 20% of articles by train click count.

Cold start is not a weakness. MIND’s cold-start users score 0.2426 against warm users’ 0.2399 with overlapping intervals, because most of the signal is article-side popularity and freshness, which needs no history. EB-NeRD has only 67 cold-start impressions (0.26% of its test split, since its users average 160 history clicks), so that interval is $21\times$ wider than the warm one and supports no conclusion either way.

The head slice is where the pipeline fails. When the clicked article is among the 20% most-clicked in *train*, EB-NeRD MRR collapses to 0.0072 against 0.2131 on the tail, and MIND falls to 0.1613 against 0.2516. The cause is the re-ranker, not Stage 1. MIND still retrieves head clicks at 91.5% (against 94.1% on the tail) yet loses a third of its MRR, and on EB-NeRD the median model rank of a *retrieved* head click is 88 of 200 against 9 on the tail. Head means most-clicked in *train*, so head clicks are old, median 145.9 h on EB-NeRD against 3.0 h, and `freshness_log_hours` carries 52% of that model’s gain, so the model demotes them by construction. It affects 0.8% of EB-NeRD impressions and 12.5% of MIND’s.

8 Metrics with and without serving-time features (Q9)

Every feature is as-of strictly before the impression it scores, so nothing reads the future. That is necessary but not sufficient: a feature can be leak-free and still be unavailable to a live system. Two features fail that test.

`impression_size`, the length of the impression’s own in-view list. On EB-NeRD that list grows with how long the reader stays on the page, and correlates with the same impression’s `read_time` at Spearman **0.50**, so it is measured from an outcome. On MIND it is the platform’s list length, which a system choosing its own candidates is never handed.

`has_known_publish_time`, read off the dataset’s later `published_time` snapshot. No live system could know it.

A stricter arm also drops `session_clicks_before`, which a session store would mostly hold but

which applies no click reporting lag. Arms differ only in the feature list; the `all` arm reproduces the shipped report to within 10^{-14} .

	Arm	Feat.	AUC	MRR	nDCG@5	nDCG@10
EB-NeRD	stage 1 only	—	0.8815	0.1639	0.1471	0.1994
	all	24	0.9219	0.2115	0.2026	0.2705
	serving-safe	22	0.9102	0.1897	0.1754	0.2372
	serving-safe (strict)	21	0.9103	0.1887	0.1739	0.2369
MIND	stage 1 only	—	0.8655	0.2237	0.2451	0.2932
	all	24	0.8891	0.2403	0.2618	0.3150
	serving-safe	22	0.8866	0.2397	0.2619	0.3133
	serving-safe (strict)	21	0.8859	0.2375	0.2599	0.3112

Table 6: Test split, popular candidates. Serving-safe drops `impression_size` and `has_known_publish_time`; strict also drops `session_clicks_before`.

EB-NeRD’s headline nearly halves. Its gain over stage 1 falls from +0.0476 MRR [+0.0442, +0.0510] to +0.0258 [+0.0229, +0.0287]. The honest claim is +0.0258. **MIND’s holds**, +0.0167 [+0.0149, +0.0183] to +0.0160 [+0.0145, +0.0175], because it has the history-similarity features to fall back on and EB-NeRD does not: dropping `impression_size` collapses EB-NeRD’s model from 174 trees to 19, MIND’s only to 108. `has_known_publish_time` carries **0.000%** of split gain on both, confirming the fresh pool already closed the future-article shortcut.

9 Design Choices

9.1 Re-Ranker Objective

The choices were pointwise *binary*, LambdaMART and the list wise *rank_xendcg*. RankNet was rejected because it minimizes the number of inversions and doesn’t treat the inversion between rank 1, 2 and rank 100, 101 differently. The pointwise is rejected because they predict the absolute relevance score of each item in isolation, completely ignoring how those items compare to each other.

9.2 NRMS improvement (Q3)

The initial idea was adding the positional encoding on the user encoder. It was rejected on measurement: at `HISTORY_SIZE` = 20, **51.1%** of MIND impressions carry fewer than 20 clicks and **29.4%** of history slots are padding, and `layers.py` applies no masking, so PE would attach position embeddings to padding on exactly the dataset where padding dominates. **Freshness** won on a category difference rather than a magnitude one: article age is not in NRMS’s input *at all* and cannot be learned from title tokens at any amount of training.

10 Source Code & AI Usage Declaration

Source Code Repository: https://github.com/18ToNyStArK18/IRE_A2

Claude Code was utilized as a coding assistant during development, with usage details documented directly in the Git repository.

11 Codabench Submissions

ID #	File name	Date	Status	Score	Detailed Results	Actions
930542	ebnrd_nrms_freshness.zip	2026-09-17 21:54	Submitted			
900883	ebnrd_submission.zip	2026-08-25 12:46	Submitted			

ID #	File name	Date	Status	Score	Detailed Results	Actions
931870	mind_nrms_baseline.zip	2026-09-18 18:01	Finished	0.6142		
930532	mind_nrms_freshness.zip	2026-09-17 21:47	Finished	0.6156		

Figure 1: Codabench leaderboard entries. Top: Ekstra Bladet News Recommendation Competition. Bottom: MIND News Recommendation Competition.